Task 1: Colour spaces & a colour picker

What each channel encodes:

HSV Channels:

1. Hue (S): Represents the pure color (0-180 in OpenCV, corresponding to 0-360 degrees), where different angles correspond to different colors (red, green, blue, etc.).
2. Saturation (S): Represents the intensity or purity of the color, from 0 (gray) to maximum (pure color).
3. Value (V): Represents the brightness of the color, from 0 (black) to maximum (full brightness).

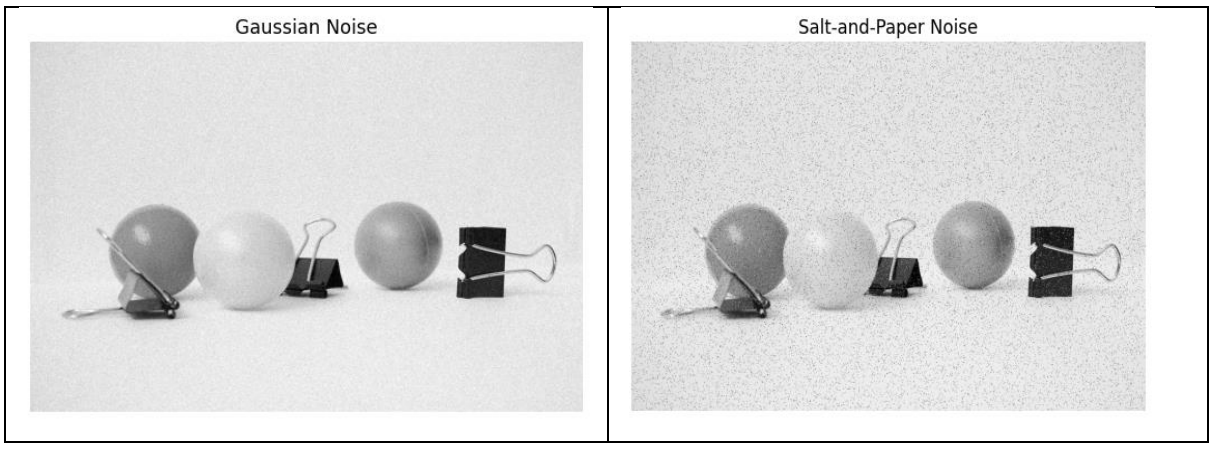
LAB Channels:

1. L (Lightness): Represents the brightness from black (0) to white (100).
2. a: Represents the color dimension from green (negative) to red (positive).
3. b: Represents the color dimension from blue (negative) to yellow (positive).

Why HSV is easier for colour-based segmentation than RGB

In RGB, a colour is defined by three values for Red, Green, Blue (RGB) that are all highly correlated with each other and with intensity changes. Even small variations in lighting or shadows can cause significant changes in all three RGB channels simultaneously. However, in HSV, the Hue channel encodes the actual colour independently of brightness and saturation. This means that regardless of lighting conditions, a red object will always have a similar hue value (around 0° or 180°), making it much easier to define a consistent range for colour segmentation. A range can be set on the Hue channel, while Saturation and Value handle the intensity variations.

Task 2: Noise and smoothing - a controlled experiment



PSNR

Filter	Noise Type	Kernel Size 3	5	7
Box	Gaussian	33.417885	34.200100	33.326508
	Salt-Pepper	25.969662	28.692699	29.438532
Gaussian	Gaussian	32.900440	34.488472	34.916885
	Salt-Pepper	25.168791	27.294378	28.867268
Median	Gaussian	31.997581	34.720791	34.983128
	Salt-Pepper	43.355595	39.762833	37.287071

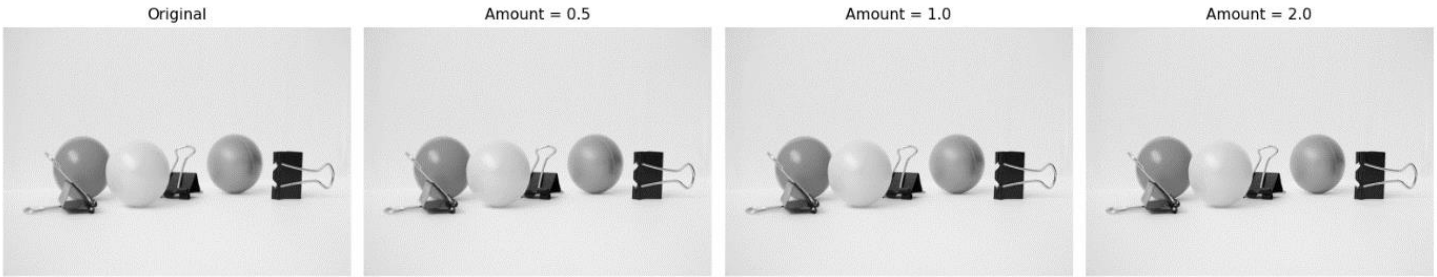
From the PSNR results:

For Gaussian noise, the Gaussian filter performs best because its smoothing kernel matches the statistical distribution of the noise which allows it to reduce the random fluctuations effectively while preserving the underlying image structure. The median filter also performs reasonably well, but the Gaussian filter is slightly more consistent across kernel sizes for this type of noise.

For salt-and-pepper noise, the median filter performed best. Salt-and-pepper noise introduces extreme outliers by forcing pixels to black or white, and averaging filters such as box or Gaussian include those outliers in their calculations, which reduces their effectiveness. The median filter, by contrast, replaces each pixel with the median value of its neighbourhood, which naturally discards those extreme values. This makes it far more effective at restoring images corrupted by impulsive noise.

Task 3: Sharpening with unsharp masking

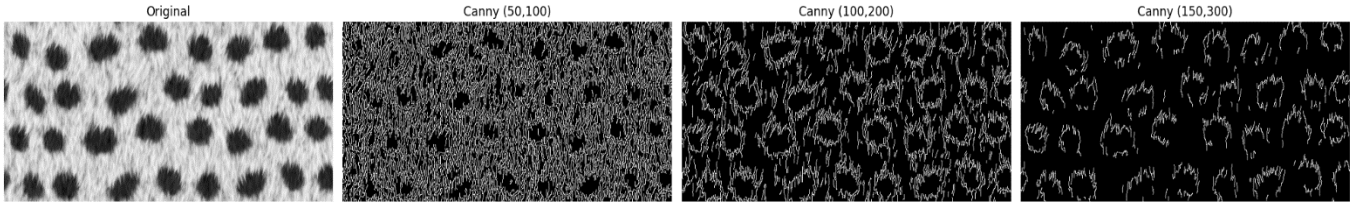
Unsharp Masking



Task 4. Edge detection three ways

Sobel – 2D Convolution produced max absolute difference of 0.0

Images for Canny detection with different thresholds

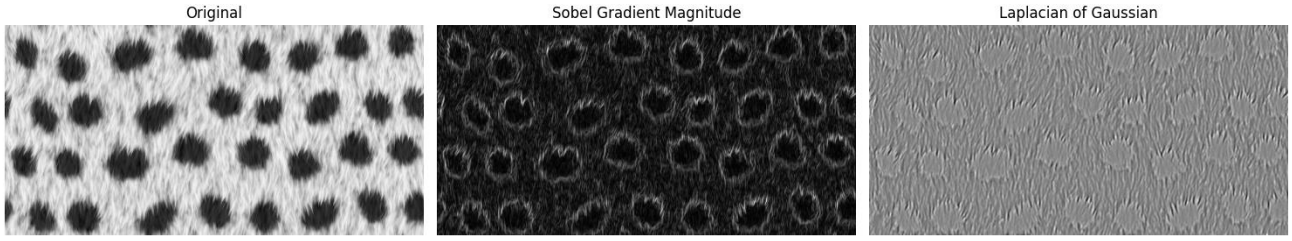


Effects of the thresholds are as follows:

i. Thresholds (50, 100): Many edges, including weak ones and some noise, are kept.
ii. Thresholds (100, 200): Balanced map: strong edges are kept while suppressing weak isolated edges.
iii. Thresholds (150, 300): Only the strongest edges remain, giving a cleaner but sparser map.

This demonstrates the role of the two thresholds in hysteresis: the high threshold ensures only strong edges are kept, while the low threshold allows weak edges to survive if they connect to strong ones.

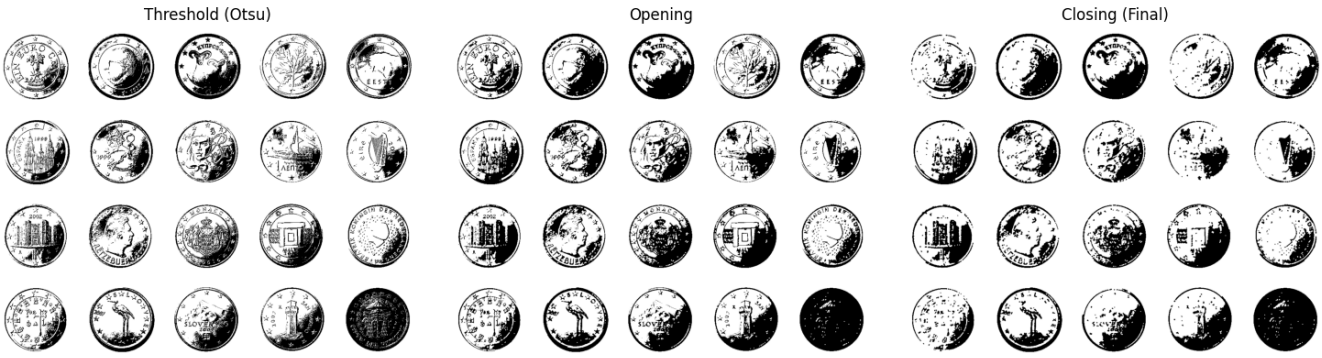
Images for Laplacian of Gaussian (LoG)



Comparison (Sobel vs LoG)

The Sobel operator produces edge maps that seem to emphasise the strength and direction of intensity changes, as in the picture above, while the Laplacian of Gaussian has thinner, symmetric outlines. This difference in character reflects Sobel's role as a first-derivative detector and LoG's role as a second-derivative detector.

Task 5: Morphology: from mask to object count



The 3×3 elliptical kernel was chosen because it balances noise removal with object preservation: it cleans the mask without merging coins. The minimum area filter of 70 pixels ensures that only blobs large enough to be coins are counted, eliminating hundreds of tiny fragments.

Task 6: Assemble a pipeline tool + reflection

Pipeline tool is the file **process.py** in the project folder